3. Hafta

Sınıflar: Kapsülleme ve `Kitap` Sınıfı

◎ Öğrenme Hedefleri

Bu bölümü tamamladığınızda:

- Bir struct + serbest fonksiyon tasarımının veri bütünlüğünü neden koruyamadığını açıklayabileceksiniz;
- `public`/`private` erişim belirleyicilerini ve kapsülleme (encapsulation) ilkesini uygulayabileceksiniz;
- Parametreli kurucu, üye başlatma listesi ve yıkıcı yazabilecek; `this` işaretçisinin ne işe yaradığını açıklayabileceksiniz;
- `Kitap` sınıfını private alanlar ve public arayüzle tasarlayıp uygulayabileceksiniz;
- Bir sınıfı `.hpp`/`.cpp` olarak ayırıp `namespace` içinde derlenebilir bir kütüphane parçası olarak kurabileceksiniz.

3.1 Neden Sınıf? Struct'tan Kapsüllemeye

1 • Ön-öğretim **Neden Önemli?**

C'deki bir `struct Kitap` tasarımında — yani alanları yan yana, açıkta duran bir veri paketinde — programınızın herhangi bir yerinden gelen bir satır `k.yil = -1500;` yazabilir; hiçbir şey buna engel olmaz. Proje büyüyüp onlarca fonksiyon aynı struct'a dokundukça, "kim, ne zaman, hangi alanı geçersiz bir değere düşürdü?" sorusunun izini sürmek neredeyse imkânsızlaşır. Bu ünite, veriyi kimin nasıl değiştirebileceğine artık serbest fonksiyonların değil, tipin kendisinin karar vermesini sağlar.

1 • Ön-öğretim **Ön Bilgi**

Bu üniteye girmeden önce Hafta 2'den şunları hatırlayın: C `struct` tanımı ve kullanımı, serbest fonksiyon tanımı ile çağrısı, temel çıktı (`cout`). Bu ünite tam da bu "struct + serbest fonksiyon" modelinin üstüne kurulur: aynı veriyi ve o veri üzerindeki davranışı, erişim kontrolüyle tek bir çatı altında toplar. Dolayısıyla bu kavramların taze olması gerekir.

Geçen hafta bellek ve fonksiyonlarla çalıştınız: bir işaretçinin gösterdiği veriyi bir fonksiyona geçirdiniz, o fonksiyon veriyi okudu ya da değiştirdi. Bu haftaya kadar kullandığınız C tarzı tasarımda veri ile o veri üzerindeki davranış birbirinden ayrı yaşar: bir yanda alanları tutan bir `struct`, öte yanda o `struct`'ı parametre alan *serbest* fonksiyonlar. `struct`, birbiriyle ilgili birkaç değeri tek bir isim altında toplar; fonksiyonlar ise o değerler üzerinde ne yapılacağını ayrı ayrı tanımlar. Örneğin bir kitabı temsil etmek isteseniz şöyle bir şey yazardınız:

```
struct Kitap {
   std::string baslik;
   std::string yazar;
   std::string isbn;
   int yil;
};

void kitapYazdir(const Kitap& k) {
   std::cout << k.baslik << " (" << k.yil << ")\n";
}
```

Bu tasarım çalışır ve küçük programlarda gayet iş görür. Ama içinde sessiz bir kusur taşır ve bu kusuru görmek için soyut kalmadan tek bir satırı adım adım izlemek yeterlidir.

3.1.1 Struct neden veriyi koruyamaz?

Diyelim ki kod tabanının çok uzak bir köşesinde, sizin hiç aklınızda olmayan bir fonksiyon şu satırı çalıştırıyor:

```
k.yil = -3; // eksi bir basim yili: anlamsiz ama tamamen "gecerli"
```

Şimdi bu satırın başına ne geldiğini sırayla düşünün. **Birincisi**, derleyici bu satıra hiç itiraz etmez: `yil` bir `int` alandır, `-3` de geçerli bir `int` değeridir, dolayısıyla bu, tip açısından kusursuz bir atamadır. `struct`'ın içinde "`yil` pozitif olmalı" diyen hiçbir kural yoktur, çünkü bir `struct` kendi başına hiçbir kod barındırmaz — yalnızca yan yana dizilmiş alanlardan oluşan, edilgen bir veri paketidir. Kural koyabilecek bir gövdesi olmadığı için, hiçbir şeyi denetleyemez. **İkincisi**, atama sessizce başarılı olur: ne bir hata, ne bir uyarı. Programınız çalışmaya kaldığı yerden devam eder — ama artık elinizde eksi üçüncü yılda basılmış, gerçek dünyada karşılığı olmayan bir "kitap" vardır. Nesne artık *tutarsız* bir durumdadır, üstelik bunu kimse fark etmemiştir.

Üçüncüsü — ve en sinsi kısmı — hata tam da bu satırda patlamaz. Bozuk değer bellekte sessizce bekler. Program dakikalarca, belki gün boyu sorunsuz akar; ta ki bambaşka bir yerde bir başka fonksiyon o alanı *okuyana* kadar: kitabın yaşını hesaplamak, onu yayın yılına göre sıralamak ya da ekrana basmak için. Yanlış davranış işte orada, o okuma noktasında yüzeye çıkar. Yani hatanın *belirdiği* yer ile hatanın *doğduğu* yer birbirinden hem zaman hem de kod konumu olarak kopuktur.

Bu kopukluk küçük bir programda önemsizdir — yüz satırlık bir kodda suçluyu birkaç dakikada bulursunuz. Ama binlerce satırlık, birden çok kişinin birlikte yazdığı bir kod tabanında durum tersine döner. Belirtiyi (çöp çıktı ya da çöken program) okuma noktasında görürsünüz, ama sebep — o `k.yil = -3;` satırı — belki binlerce satır ve onlarca dosya ötededir. `Kitap`'a dokunan yüzlerce fonksiyondan *herhangi biri* bu değeri bozmuş olabileceği için, aslında hepsini birer birer gözden geçirmeniz gerekir. İşte bir alanı "açıkta" bırakmanın gerçek bedeli budur: hata ucuz üretilir, ama pahalı bulunur.

Bütün bunların altındaki kök neden tek cümleyle şudur: "bu alana kim, nasıl dokunabilir?" sorusunun cevabı yalnızca programcıların disiplinine, yazılı olmayan

bir uzlaşıya kalmıştır. Küçük ölçekte disiplin yeter; ölçek büyüdükçe, birileri er ya da geç unutur. İhtiyacınız olan şey, bu kuralı bir uzlaşı olmaktan çıkarıp dilin *kendi* zorunlu kıldığı bir şeye çevirmektir — yani veriyi, ona dokunma kurallarıyla birlikte tek bir yere kilitlemek.

3.1.2 Veri ile davranışı bir araya getirmek

C++'ın `class` anahtar kelimesi bu sorunu farklı bir tasarım ilkesiyle çözer: veriyi ve o veri üzerinde çalışan davranışı aynı tanımın içine, birlikte yerleştirir; üstelik veriye kimin nasıl erişebileceğini de aynı tanım belirler. Böylece "kural" artık programcının hatırlamasına kalan bir uzlaşı değil, sınıfın tanımına gömülü, derleyicinin zorla uyguladığı bir yapı hâline gelir.

Tanım: Sınıf ve nesne

Bir *sınıf* (*class*), veriyi (alanlar) ve o veri üzerinde çalışan davranışı (üye fonksiyonlar) tek bir kullanıcı tipi altında bir arada tanımlar; ayrıca bu alanlara sınıf dışından kimin erişebileceğini de kendisi belirler. Bir *nesne* (*object*) ise bir sınıftan üretilmiş somut bir örnektir — tıpkı `struct Kitap` tanımından bir `Kitap` değişkeni ürettiğiniz gibi, `class Kitap` tanımından da bir `Kitap` nesnesi üretirsiniz. Aradaki fark tanımın kendisinde: sınıf, verinin yanına artık davranışı *ve* bir erişim kuralını da katar.

kapsülleme: iç durumu (verileri) dışarıdan doğrudan erişilemez kılıp, yalnızca sınıfın sunduğu kontrollü bir arayüzden eriştirme ilkesi (*encapsulation*).

Bu birleşim ilkesinin adı **kapsüllemedir**: sınıf kendi iç durumunu gizler, dışarıya yalnızca izin verdiği kadarını, izin verdiği biçimde açar. Struct + serbest fonksiyon tasarımında veri tamamen açıktaydı ve az önce gördüğünüz gibi herhangi bir satır onu sessizce bozabiliyordu; kapsülleme ile veri sınıfın kendi denetimine girer, dolayısıyla "`yil` pozitif olmalı" gibi bir kural artık verinin *yanında*, ondan hiç ayrılmadan durabilir.

Analoji

Kapsüllemeyi bir otomat makinesine benzetebilirsiniz. Bir otomatın önünde durduğunuzda içindeki ürünlere, raflara ya da para kutusuna elle uzanamazsınız; makineyle ilişkiniz tamamen dıştaki birkaç düğme ve para yuvası üzerinden yürür. Para atarsınız, bir tuşa basarsınız, makine size seçtiğiniz ürünü ve para üstünü verir. Stoğun ne kadar azaldığını, kasadaki paranın ne kadar arttığını makine kendi içinde, siz hiç görmeden yönetir. Camı kırıp içeri uzanmak — yani arayüzü atlayıp iç düzene doğrudan müdahale etmek — makinenin bütün kurallarını çiğnemek, onu tutarsız bir duruma sokmak olur; zaten tasarım da tam olarak bunu engellemek için oradadır.
Şimdi bu benzetmeyi doğrudan koda eşleyin. Para yuvası ve tuşlar, sınıfın *dışarıya açtığı* kontrollü arayüzdür — birazdan göreceğiniz `public` üyeler. İçteki stok ve kasa, makinenin *gizli iç durumudur* — sınıfın `private` alanları. Camı kırıp içeri uzanmak ise bir `private` alana dışarıdan doğrudan erişmeye karşılık gelir; kapsülleme, tam olarak bu "camı kırma" hamlesini baştan imkânsız kılar. (Aynı fikri bir bankamatikte ya da bir TV kumandasında da görürsünüz: kasaya elinizi sokmadan, birkaç düğmeyle işinizi görürsünüz; iç durum daima gizli, etkileşim daima kontrollü bir arayüzden.)

3.1.3 `public` ve `private`: erişim belirleyiciler

erişim belirleyici: bir üyenin (alan ya da üye fonksiyon) sınıf dışından görünür olup olmadığını belirleyen anahtar kelime (*access specifier*); en sık kullanılan ikisi `public` ve `private`'tır.

Bu kontrolü sağlayan araç, sınıf gövdesine yazdığınız **erişim belirleyici** anahtar kelimelerdir. Bir erişim belirleyici, kendinden sonra gelen üyelerin görünürlüğünü belirler ve bir sonraki belirleyiciye kadar geçerli kalır. Kuralları iki cümlede toplanabilir: `public` altında duran üyelere sınıfın dışından *herhangi* bir kod erişebilir; `private` altında duranlara ise *yalnızca* sınıfın kendi üye fonksiyonları erişebilir, dışarıdaki hiçbir kod erişemez.

Bu iki cümlenin pratikte ne anlama geldiğini somut bir çağrı üzerinden görün. Elinizde bir `k` nesnesi olduğunu varsayın. Eğer `yil` alanı `public` ise, dışarıdaki herhangi bir kod `k.yil = -3;` yazabilir ve derleyici buna izin verir — yani bir önceki bölümdeki sorunun kapısı hâlâ açıktır. Eğer aynı alan (`yil_` adıyla) `private` ise, dışarıdan `k.yil_ = -3;` yazmayı denediğiniz anda program *derlenmez*: derleyici size "bu üye `private`, buradan erişemezsiniz" der ve hatayı daha ilk derlemede, kimse çöp bir değerle karşılaşmadan önce yakalar. Kısacası `public`, "buraya herkes dokunabilir" demenin; `private` ise "buraya yalnızca sınıf dokunabilir" demenin dilidir.

Aşağıdaki küçük örnek, aynı `Kitap` verisinin C tarzı bir `struct` olarak mı, yoksa kapsülleyen bir `class` olarak mı tanımlandığını yan yana gösteriyor:

```
// C tarzi: tum alanlar acikta, kim isterse degistirir
struct KitapStruct {
   int yil; // struct'ta varsayilan erisim: public
};

// C++ tarzi: alan gizli, degisim yalniz kontrollu arayuzden
class KitapClass {
public:
   void setYil(int yil) { if (yil > 0) yil_ = yil; } // kural burada
private:
   int yil_; // class'ta varsayilan erisim: private
};
```

`KitapStruct` sürümünde herhangi bir kod `k.yil = -3;` yazıp alanı doğrudan bozabilir; struct'ın buna diyecek bir sözü yoktur. `KitapClass` sürümünde ise `yil_` alanı `private` olduğu için sınıf dışından görünmez bile — dolayısıyla dışarıdan doğrudan atanamaz. Alanı güncellemenin tek yolu `setYil()` üye fonksiyonundan geçmektir, ve tam da bu fonksiyonun içine `yil > 0` gibi bir doğrulama kuralı yerleştirebilirsiniz. Adım adım izleyin: dışarıdan biri `k.setYil(-3)` çağırdığında, `if (yil > 0)` koşulu `-3` için yanlış olur, atama hiç yapılmaz ve `yil_` eski (geçerli) değerinde kalır; `k.setYil(2021)` çağırdığında ise koşul doğru olur ve alan güncellenir. Böylece "geçersiz bir yıl asla içeri giremez" kuralı, programcının hatırlamasına değil, sınıfın kendi koduna emanet edilmiş olur. Alanı `private` yapmak, işte bu yüzden "bu veriye yalnızca sınıfın belirlediği kurallarla dokunabilirsiniz" demenin dilidir.

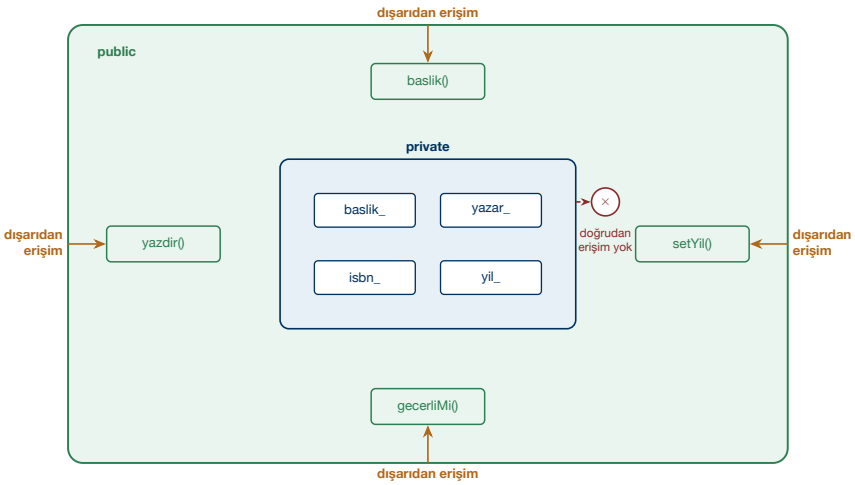
> **İpucu**
>
> "Bütün alanları `private` yapıp her biri için birer `public` okuma/yazma fonksiyonu yazsam, sonuçta yine her alana erişilmiyor mu?" diye düşünebilirsiniz. Fark inceliklidir ama belirleyicidir: kapsüllemenin amacı erişimi büsbütün kesmek değil, erişimi *kurala bağlamaktır*. `setYil()` bir alanı yazmanıza izin verir, ama yalnızca kural (`yil > 0`) sağlanırsa; bir alanı hiç dışarı açmayabilir ya da yalnızca okumaya açıp yazmaya kapatabilirsiniz. Kararı sınıf verir — dışarıdaki kod değil. Otomat da size ürün verir, ama yalnızca parayı attıktan sonra; "ver-

mek" ile "kuralsız vermek" arasındaki fark tam olarak budur.

Burada küçük ama unutulmaması gereken bir ayrıntı var: C++'ta `struct` da tıpkı `class` gibi üye fonksiyonlarına, hatta `public`/`private` bölümlerine sahip olabilir — C'deki `struct`'tan farklı olarak. İki anahtar kelime arasındaki gerçek fark yalnızca *varsayılan erişimdir*: bir üyeyi hiçbir belirleyici altında yazmadan bırakırsanız, `struct` içinde o üye varsayılan olarak `public`, `class` içinde ise varsayılan olarak `private` olur. Yukarıdaki örnek bunu bilerek gösteriyor — `KitapStruct` içindeki `yil`, hiçbir belirleyici yazmadığınız hâlde `public`'tir; `KitapClass` içindeki `yil_` ise `private:` etiketinin altında durduğu için gizlidir.

Bu haftadan itibaren kapsülleyen tipleri `class` olarak yazacaksınız, çünkü varsayılan olarak "gizli" davranış tam da istediğiniz şeydir. Nedeni pratiktir: bir alanı bir bölüm etiketi yazmayı unutarak açıkta bırakırsanız, `class` sizi `public` bir alana değil, güvenli tarafa — `private` bir alana — düşürür. Yani `class` kullanmak, dikkatsizlik anında bile sizi korur; `struct` ise tam tersine, unutkanlığı en açık (ve en tehlikeli) duruma çevirir.

Şekil 3.1 bu ayrımı görsel olarak özetliyor: içteki çekirdek `private` verinizi barındırır, onu saran kabuk ise `public` arayüzünüzdür; dışarıdan gelen her etkileşim yalnızca kabuğa değer, çekirdeğe doğrudan dokunmaz — tıpkı otomatın camına değip iç stoğuna dokunamayan bir müşteri gibi.



Şekil 3.1: Kapsülleme: `private` veri çekirdeği, yalnızca `public` arayüz kabuğu üzerinden erişilebilir olur.

Bu ünitede gördüğünüz fikir kısaca şudur: sınıf, struct'ın verisini ve serbest fonksiyonun davranışını tek bir tanım altında birleştirir; erişim belirleyiciler ise bu tanımın hangi parçasının dışarıya açık, hangisinin gizli kalacağını söyler. Böylece bir önceki bölümde izini sürdüğünüz o sessiz hata — uzaktaki bir satırın veriyi bozması — artık en baştan, derleme anında engellenir. Gelecek ünitede bu gizli veriyi bir nesne ilk kurulurken — kurucu ile — nasıl tutarlı bir başlangıç durumuna getirdiğinizi göreceksiniz.

3 • Katılım ⌨ Şimdi Siz Deneyin

Aşağıdaki iki tanımı düşünün: bir `struct A` ile bir `class B`; ikisinin de içinde tek bir `int x;` üyesi olsun. `A a; a.x = 5;` ile `B b; b.x = 5;` satırlarından hangisi derlenir, hangisi derlenmez — neden?

Geribildirim. `a.x = 5;` derlenir, çünkü `struct` içinde bir üyenin varsayılan erişimi `public`'tir. `b.x = 5;` ise derlenmez — derleyici "`x` is private within this context" der — çünkü `class` içinde varsayılan erişim `private`'tır. Bir alanı dışarıdan tutarsız değer atanmasına karşı korumak istiyorsanız, onu `private` bir sınıf üyesi yapıp yalnız kontrollü bir arayüzden eriştirirsiniz — kapsüllemenin özü tam olarak budur.

Kendinizi Sınayın 3.1

Bir sınıfta bir alanı neden `private` yaparsınız? Bir önceki bölümdeki `KitapStruct` örneğini düşünerek, alan `public` kalsaydı ne tür bir soruna yol açabileceğini de birkaç cümleyle açıklayın.

5 • İzleme — Aktarım — Başka Nerede?

"İç durumu gizle, yalnız kontrollü arayüzden eriştir" ilkesi yalnız `Kitap`'a özgü değildir: Kitaplık projesine dönem boyu eklenecek her sınıf — ör. `Uye`, `Odunc`, `Raf` — aynı desenle iç durumunu koruyacaktır. Gündelik yaşamda da aynı fikir işler: bir otomat makinesinin para kutusuna elle uzanamaz, yalnız dıştaki tuş ve para arayüzüyle etkileşirsiniz; makine kendi stok ve kasa durumunu kendisi yönetir. Bir tipin verisine "dışarıdan kim, nasıl dokunabilir?" sorusunu her yeni sınıfta sormak, bu haftanın en kalıcı alışkanlığıdır.

3.2 Kurucu ve Yıkıcı

1 • Ön-öğretim — Neden Önemli?

Ünite 1'de alanları `private` yaptınız; ama bir nesne oluşturulurken bu alanların ilk değerlerini kim, ne zaman verecek? Struct'ta ilklemeyi unutmak kolaydır — başlığı dolu ama yılı çöp değerli, yarı-tutarsız bir `Kitap` elde edersiniz. Dahası `const` ya da referans bir üyeyi sonradan hiç atayamazsınız. Bu ünite, nesneyi ilk andan tutarlı kuran kurucuyu ve ömrü bitince düzenli temizleyen yıkıcıyı tanıtır.

1 • Ön-öğretim — Ön Bilgi

Bu üniteye girmeden önce şunları hatırlayın: Ünite 1'in `private` alan ile `public` arayüz ayrımı ve kapsülleme ilkesi; Hafta 2'nin kapsam (*scope*) ile bellek/ömür sezgisi — yıkıcının "kapsam sonunda otomatik çağrılır" davranışını anlamak için buna dayanacaksınız. Bir `std::string` üyesine değer atamanın bir maliyeti (kopyalama) olduğu fikri de bu ünitede işinize yarayacaktır.

Önceki ünitede `private` alanları kapsülleyen bir sınıf tanımladınız, ama bir soruyu henüz yanıtlamadınız: bir `Kitap` nesnesi ilk var olduğu anda, alanları hangi değerlerle doludur? Sorunun önemi kapsüllemeden gelir: bir alan `private` olduğu için sınıf dışından doğrudan atanamaz; peki nesne ilk oluştuğu anda o alana *kim* bir değer koyacak? C tarzı `struct`'ta bu işi siz üstlenirdiniz — nesneyi tanımladıktan hemen sonra alan alan atama yaparak:

```
Kitap k; // dort alan da tanimsiz/copsel degerde
k.baslik = "Nutuk"; // siz unutursaniz, derleyici sizi uyarmaz
k.yil = 1927;
// yazar ve isbn atanmayi unutuldu -- kimse fark etmez
```

Bu satırları izleyin: `k` tanımlandığı anda dört alan da (özellikle `yil` gibi temel tipten olanlar) rastgele, tanımsız bir değerle belleğe yerleşir; siz aşağıdaki iki satırla yalnızca `baslik` ve `yil`'i doldurursunuz, `yazar` ve `isbn`'i atamayı unutursunuz. Derleyici bu unutkanlığa itiraz etmez — her satır tek başına geçerlidir — ama elinizde yazarı ve ISBN'i çöp değerde bir nesne kalır. Bu, Ünite 1'de gördüğünüz "alan açıkta kalırsa kural yoktur" sorununun bir başka yüzüdür: burada sorun yanlış bir değerin *sonradan* yazılması değil, hiç değer yazılmadan nesnenin *kullanılabilir* hâle gelmesidir. C++ bu görevi size bırakmaz; nesnenin kendisine, *kurucuya* devreder — kurucu çalışmadan nesne var olamaz, dolayısıyla "unutma" ihtimali kurucunun kendisinde baştan kapatılır.

3.2.1 Kurucu: nesnenin ilk anı

Tanım: Kurucu

Bir *kurucu* (*constructor*), bir sınıfla aynı adı taşıyan, dönüş tipi olmayan özel bir üye fonksiyondur; bir nesne oluşturulduğu anda derleyici tarafından *otomatik* çağrılır ve nesnenin alanlarına ilk değerlerini verir. Kurucu çalışmadan bir nesne kullanılamaz — derleyici bunu garanti eder.

"Otomatik çağrılır" ifadesini somutlaştırın: `Kitap k("Nutuk", "Mustafa Kemal", "9789751000000", 1927);` satırını yazdığınızda, derleyici bu satırı "önce boş bir `k` oluştur, sonra kuruculuğunu elle çağır" biçiminde okumaz. Derleyici, `k` adının bellekte yer ayırdığı anla kurucunun çalıştığı anı *aynı satıra* sıkıştırır — siz `k.Kitap(...)` gibi ayrı bir çağrı yazamazsınız bile (kurucunun adı yoktur, doğrudan çağrılamaz). Bu, `struct`'taki alan-alan atamadan köklü bir farktır: orada nesne önce var olur, sonra siz onu (isterseniz, unutabileceğiniz biçimde) doldurursunuz; burada nesnenin var olması ile doldurulması aynı, bölünemez adımdır.

parametreli kurucu: çağrıldığında değer alan, bu değerlerle nesneyi kuran kurucu (*parameterized constructor*).

Bir sınıfın birden çok kurucusu olabilir, yeter ki parametre listeleri birbirinden ayırt edilebilsin (aşırı yükleme). En sık karşılaştığınız ikisi şunlardır: **parametreli kurucu** ile hiç parametre almayan *varsayılan kurucu* (*default constructor*). Sınıfınızda hiçbir kurucu tanımlamazsanız derleyici size boş bir varsayılan kurucu üretir; ama bu üretilmiş kurucu alanları *ilklemez*, yalnızca "kurucu çağrılabilir" der — temel tipten (`int`, işaretçi) bir alan böyle bırakılırsa çöp bir değerle kalır, tıpkı yukarıdaki `struct` örneğindeki gibi. `Kitap` gibi her zaman geçerli bir başlığı, yazarı, ISBN'i ve yılı olması gereken bir sınıf için varsayılan kurucuya güvenmek anlamsızdır: siz kendi parametreli kurucunuzu yazarsınız ki nesne ancak gerekli tüm bilgilerle birlikte var olabilsin — eksik bilgiyle bir `Kitap` nesnesi kurmak, derleme aşamasında imkânsız hâle gelsin.

3.2.2 Üye başlatma listesi mi, gövdede atama mı?

Kurucunun gövdesi içinde alanlara değer atayabilirsiniz — tıpkı bir `struct` için yaptığınız gibi:

```
Kitap::Kitap(std::string baslik, int yil) {
    baslik_ = baslik; // govdede atama: uye once ilklenir, sonra uzerine
        yazilir
    yil_ = yil;
```

```
}
```

Bu kodun çalışma zamanında *tam olarak* ne yaptığını iki adıma ayırarak izleyin. **Birinci adım**, gövde hiç çalışmadan *önce* gerçekleşir ve siz onu görmezsiniz: derleyici, kurucu gövdesine girmeden hemen önce her üyeyi kendi varsayılan biçiminde ilkler — `baslik_` bir `std::string` olduğu için burada `std::string`'in kendi varsayılan kurucusu çalışır ve boş bir dizge üretir. **İkinci adım** gövdenin kendisidir: `baslik_ = baslik;` satırı çalışır ve az önce kurulmuş olan boş dizgenin *üzerine* parametre değerini yazar. Sonuçta `baslik_` doğru değerde biter, ama oraya iki işlemle — önce kur, sonra üzerine yaz — ulaşır; ilk kurma işi tamamen israf edilmiş bir emektir, çünkü sonucu hiç kullanılmadan bir sonraki satırda çöpe atılır.

üye başlatma listesi: kurucu gövdesinden önce, `:` işaretiyle başlayıp üyeleri gövde hiç çalışmadan doğrudan ilkleyen sözdizimi (*member initializer list*).

C++'ın sunduğu ikinci — ve tercih edilen — yol, kurucu imzasından sonra, gövdeden önce gelen **üye başlatma listesi** yoludur:

```
Kitap::Kitap(std::string baslik, int yil)
   : baslik_(baslik), yil_(yil) {} // uye baslatma listesi: tek adimda
       ilklenir
```

Aynı çalışma zamanı sırasını bu sürüm için de adım adım izleyin. Derleyici, gövdeye girmeden önce yine her üyeyi ilkler — ama bu kez "önce varsayılan, sonra üzerine yaz" değil, doğrudan `: baslik_(baslik), yil_(yil)` listesindeki değerle ilkler. `baslik_` hiçbir zaman "önce boş, sonra dolu" aşamasından geçmez; `baslik` parametresinin değeriyle *doğrudan* kurulur, çünkü burada çalışan `std::string`'in varsayılan kurucusu değil, onun kopyalayan (değerden kuran) kurucusudur. Gövde bu sürümde zaten boştur (`{}`) — yapılacak ek bir iş kalmamıştır, çünkü ilkleme işi listede tamamen bitmiştir.

Basit tipler (`int` gibi) için bu iki yol arasındaki fark gözle görünmeyebilir — bir `int`'i "önce ilkle sonra üzerine yaz" işlemi ile "doğrudan ilkle" işlemi neredeyse aynı maliyettedir. Ama `std::string` gibi kendi kurucusu olan bir üyede fark somuttur: gövdede atama yaklaşımı üyeyi önce (varsayılan kurucusuyla) kurup sonra (atama operatörüyle) yeniden doldurur — *iki* kurucu/operatör çağrısı; üye başlatma listesi ise aynı işi *tek* bir kurucu çağrısına indirir. Sınıfınızda çok sayıda `std::string` ya da benzer üye varsa, bu fark her nesne kuruluşunda tekrarlanan, gereksiz bir maliyettir.

Daha da önemlisi, bazı üye türleri için üye başlatma listesi bir tercih değil, bir *zorunluluktur*. Nedenini küçük bir örnekle görün:

```
class Ornek {
   const int sabit_; // const uye: yalnizca ilklenebilir
public:
   Ornek(int deger) {
      sabit_ = deger; // DERLEME HATASI: const uyeye atama yapilamaz
   }
};
```

Bu kod derlenmez, çünkü `const` bir üye bir kez ilklendikten sonra bir daha *hiçbir zaman* değiştirilemez — oysa gövdedeki `sabit_ = deger;` satırı tam olarak bunu, ilklenmiş bir üyeyi değiştirmeyi dener (üstelik üye zaten gövdeye girmeden önce ilklenmiş olduğundan, bu satır bir "ilk atama" değil bir "yeniden atama" sayılır). Aynı kısıt referans üyeler için de geçerlidir: bir referans hangi nesneyi gösterdiğini yalnızca kurulduğu anda alır, sonradan başka bir nesneyi göstermek üzere değiştirilemez. Böyle bir üyeniz varsa gövdede atama seçeneği baştan yoktur — tek yol üye başlatma listesinde `: sabit_(deger)` biçiminde doğrudan ilklemektir.

Gerçek `Kitap` kurucusu tam olarak bu deseni izler — dört alanın tümü üye başlatma listesinde ilklenir, gövde boş kalır:

```
6 Kitap::Kitap(std::string baslik, std::string yazar, std::string isbn, int yil
      )
7     : baslik_(std::move(baslik)), yazar_(std::move(yazar)),
8       isbn_(std::move(isbn)), yil_(yil) {}
```

Bu satırı yukarıdaki adımlarla aynı sırayla okuyun: `Kitap k("Nutuk", "Mustafa Kemal", "9789751000000", 1927);` çalıştığında derleyici önce dört yerel parametreyi (`baslik`, `yazar`, `isbn`, `yil`) kurar, ardından gövdeye girmeden `:` listesini soldan sağa işler — `baslik_` `baslik`'tan, `yazar_` `yazar`'dan, `isbn_` `isbn`'den, `yil_` `yil`'dan doğrudan kurulur — ve en sonda boş gövde (`{}`) hiçbir ek iş yapmadan çalışır. Bu satır tamamlandığında `k` artık dört alanı da dolu, tutarlı bir nesnedir; hiçbir alan "önce boş" aşamasından geçmiş değildir.

`std::move` kullanımını burada bir ayrıntı olarak not edin: kurucuya değer olarak geçirilen `baslik`, `yazar` ve `isbn` parametreleri zaten kurucunun kendi yerel kopyalarıdır; bunları `baslik_` gibi üyelere *taşımak*, gereksiz bir ek kopyalamadan kaçınır. Bu taşıma konusuna ileride daha derinlemesine döneceksiniz; şimdilik üye başlatma listesinin genel deseninin ne olduğunu görmeniz yeterli.

Analoji

Bir kütüphaneye yeni bir üye kaydı açtığınızı düşünün. Kayıt formunu doldurmadan — ad, soyad, üyelik numarası — kimse sisteme "yarım" bir üye olarak girilmez; kayıt masası formu tamamlanmadan işlemi başlatmaz bile, ve form bir kere onaylandıktan sonra üzerindeki üyelik numarası bir daha değiştirilmez. Kurucu da tam olarak budur: nesneyi hiçbir zaman "kayıtsız" bir ara durumda bırakmaz, ilk andan itibaren tutarlı bir başlangıç durumunda kurar.
Şimdi bu benzetmeyi doğrudan koda eşleyin. Üye başlatma listesi, kayıt görevlisinin formdaki her alanı — ad, soyad, üyelik numarası — tek bir oturuşta, *doğrudan* doldurmasına karşılık gelir: form hiçbir zaman yarı dolu bir hâlde ortada kalmaz. Gövdede atama ise görevlinin önce boş bir form basıp elinize tutuşturmasına, sonra siz doldurunca formu yeniden toplayıp bilgileri sisteme geçirmesine benzer — sonuç aynı görünse de araya bir gereksiz adım (boş formu basmak) girmiştir. `const` bir üye, tıpkı forma bir kere yazılıp mühürlenen üyelik numarası gibidir: kayıt anında bir kere yazılır, o andan sonra üzerinde oynama yapılamaz — dolayısıyla böyle bir alanı "önce boş bas, sonra doldur" yoluyla vermeniz mümkün değildir, doğrudan doldurmanız gerekir.

3.2.3 `this` işaretçisi

Bir üye fonksiyon çalıştığında, hangi nesne üzerinde çalıştığını nasıl bilir? Somut bir soruyla başlayın: elinizde `Kitap a` ve `Kitap b` gibi iki ayrı nesne varsa, `a.yazdir();` çağrısı `a`'nın alanlarını mı, `b`'nin alanlarını mı basmalıdır? Cevap açıktır — `a`'nınkileri — ama `yazdir()` fonksiyonunun gövdesindeki `baslik_` yazımı, kendi başına hangi nesnenin `baslik_`'i olduğunu söylemez. Bu boşluğu dolduran şey, derleyicinin her üye fonksiyona örtük olarak geçirdiği **this** adlı işaretçidir; `this` her zaman "bu üye fonksiyonun üzerinde çağrıldığı nesnenin adresi"ni tutar. `a.yazdir()` çağrısında `this`, `&a`'ya eşittir; `b.yazdir()` çağrısında ise aynı gövde, aynı kod, ama `this` bu kez `&b`'ye eşittir — gövdedeki `baslik_` ifadesi derleyici tarafından örtük olarak `this->baslik_` diye okunduğu için, hangi çağrıda hangi nesnenin alanına eriştiği otomatik olarak doğru çıkar.

this: bir üye fonksiyonun çağrıldığı nesneye örtük olarak sahip olduğu, o nesnenin adresini tutan işaretçi (*implicit pointer to the calling object*).

Çoğu zaman `this`'i açıkça yazmanız gerekmez — `yil_` yazdığınızda derleyici bunu zaten `this->yil_` olarak okur — ama bir parametre adı bir üye adıyla çakış-

tığında `this` çakışmayı çözmenin yoludur. Şu kısa gövdeyi düşünün:

```
void setYil(int yil) { this->yil_ = yil; }
```

Burada iki farklı "yıl" bir aradadır: parametre `yil` ve üye `yil_`; adlandırma kuralı gereği sondaki alt çizgi zaten bu ikisini görsel olarak ayırır, ama `this->yil_` yazımı ayrıca "sol taraftaki bu nesnenin üyesi, sağ taraftaki gelen parametre" ayrımını derleyiciye de açıkça belirtir. `this->yil_ = yil;` ifadesinde sol taraftaki `yil_` nesnenin üyesini, sağ taraftaki `yil` ise parametreyi gösterir.

3.2.4 Yıkıcı: nesnenin son anı

yıkıcı: nesnenin ömrü bitince derleyicinin otomatik çağırdığı, son toparlama işini yapan özel üye fonksiyon (*destructor*).

Kurucu bir nesnenin doğuşunu kurduğu gibi, **yıkıcı** da nesnenin ömrünün bitişini kurar. Yıkıcı, sınıf adının başına `~` işareti eklenerek yazılır (`~Kitap()`), parametre almaz ve derleyici tarafından *otomatik* çağrılır — onu siz elle çağırmazsınız (kurucunun aksine yıkıcının hiç parametresi yoktur, çünkü "nasıl yok edileceği" konusunda seçim şansınız yoktur: her nesne aynı biçimde yıkılır).

Ne zaman çalıştığını somut bir örnekle izleyin:

```
void islemYap() {
   Kitap k("Nutuk", "Mustafa Kemal", "9789751000000", 1927);
   k.yazdir();
} // <-- k burada kapsam disina cikar, yikici burada calisir
```

`islemYap()` çağrıldığında sırasıyla şunlar olur: `k` tanımlanır ve kurucusu çalışır; `k.yazdir()` çağrılır ve ekrana basar; fonksiyon gövdesinin kapanış parantezine gelinir. Tam bu son satırda — siz hiçbir şey yazmadan — derleyici `k` için yıkıcıyı otomatik çağırır, çünkü `k` yerel bir değişken olarak tanımlanmıştı ve onun kapsamı (*scope*) burada, fonksiyon gövdesinin sonunda biter. Bir yerel değişken olarak oluşturulan bir nesnenin yıkıcısı her zaman böyle çalışır: nesne kapsam dışına çıktığı anda, tanımlandığı bloğun kapanışında tetiklenir. Aynı kural iç içe bloklar için de geçerlidir: bir `if` gövdesi içinde tanımlanan bir `Kitap` nesnesi, o `if` bloğu kapanır kapanmaz yıkılır — fonksiyonun tamamı bitmesini beklemez.

İpucu

Bu haftaki `Kitap` sınıfı yalnızca kendi içine gömülü `std::string` ve `int` üyeler tutuyor; bunların kendi yıkıcıları zaten kendi kaynaklarını (dizge belleği gibi) otomatik serbest bırakıyor, bu yüzden `Kitap` için ayrı bir yıkıcı yazmanıza bile gerek yok. Ama bir sınıf `new` ile ayrılmış ham bellek ya da açık bir dosya tanıtıcısı gibi bir *kaynağa* sahip olsaydı, o kaynağı serbest bırakma sorumluluğu tam olarak yıkıcıya düşerdi. Gelecek hafta bu soruyu derinlemesine ele alacaksınız: bir nesne bir kaynağa sahipken bu kaynağı hatasız, sızıntısız biçimde nasıl yönetirsiniz — akıllı işaretçiler tam olarak bu ihtiyaçtan doğar.

Örnek Olay 3.1: Bir `Kitap` nesnesinin yaşam döngüsü

Aşağıdaki adımları bir `main` fonksiyonu içinde çalıştırdığınızı düşünün:

1. **Oluşturma:** `Kitap k("Nutuk", "Mustafa Kemal", "9789751000000", 1927);` satırı çalıştığında derleyici otomatik olarak kurucuyu çağırır; üye başlatma listesi dört alanı da tek adımda, hiçbiri "önce boş" aşamasından geçmeden ilkler. Bu satır tamamlandığı anda `k` artık tutarlı, geçerli bir nesnedir — hiçbir alan boş veya rastgele bir değerde değildir; nesne var olmadan önce "kısmen dolu" bir hâlde gözlemlenemez.

2. **Kullanma:** `k.yazdir();` çağrısı `k`'nin üyelerini okuyup ekrana basar; her çağrıda derleyici örtük olarak `this`'i `&k` yapıp gövdeye geçirir, böylece `baslik_` gibi ifadeler doğru nesnenin alanını gösterir. `k.setYil(1934);` gibi bir çağrı ise `yil_` alanını doğrudan bir atama yerine kontrollü bir arayüzden günceller — `setYil` gövdesindeki doğrulama (yıl pozitif mi) çalışır, sonra `this->yil_` güncellenir. Bu adımlar boyunca `k` yaşamaya, kullanılmaya devam eder; kurucu yalnızca bir kez, en başta çalışır.

3. **Yıkım:** `main` fonksiyonu sonuna geldiğinde ya da `k`'nin tanımlı olduğu blok kapandığında, derleyici `k` için yıkıcıyı otomatik çağırır — siz `k.~Kitap();` gibi bir şey yazmazsınız, bu tamamen örtük gerçekleşir. Yıkıcı çalıştıktan sonra `k` artık yoktur; `k`'ye ait bellek geri kazanılır ve aynı ada bir daha erişmek programın davranışını tanımsız kılar.

Bu üç adım — otomatik kuruluş, kontrollü kullanım, otomatik yıkım — her `Kitap` nesnesinin hayatının aynı kalıbıdır; kurucu ve yıkıcı bu kalıbın başını ve sonunu, siz hiçbir ek satır yazmadan garanti eder. İlk adımın kurucuyla, son adımın yıkıcıyla otomatik gerçekleşmesi, ortadaki kullanım adımının her zaman tutarlı, geçerli bir nesne üzerinde yürüdüğünü de garanti eder — çünkü nesne ancak tam kurulduktan sonra var olabilir, yıkılana kadar da var olmaya devam eder.

Ortak Hata / Tuzak

En sık düşülen tuzak, üye başlatma listesini hiç kullanmayıp tüm ilklemeyi kurucu gövdesine, atama olarak yazmaktır. Küçük sınıflarda bu "çalışır" görünür, ama üç somut sorunu beraberinde getirir. (1) **Gereksiz çift işlem:** yukarıda adım adım gördüğünüz gibi üye önce (varsayılan kurucusuyla) ilklenir, sonra gövdede üzerine yazılır — `std::string` gibi üyelerde bu iki ayrı işlem demektir, ve sınıfınızda çok sayıda böyle üye varsa maliyet her nesne kuruluşunda tekrarlanır. (2) **Derleyici hatası:** `const` ya da referans bir üyeniz olduğunda gövdede atama seçeneği baştan yoktur — yukarıdaki `Ornek` sınıfında gördüğünüz gibi, böyle bir üye gövdede *atanamaz*, yalnız üye başlatma listesinde ilklenebilir; siz gövdede atamayı denerseniz kod hiç derlenmez. (3) **Sessiz unutkanlık:** bir üyeyi hem başlatma listesinde hem gövdede unutmak — eğer bir alanı ne listede ne gövdede ilklerseniz, o alan temel bir tipse (`int`, işaretçi) tanımsız/çöp bir değerle sessizce kalır — derleyici sizi *her zaman* uyarmaz, tıpkı bu ünitenin başındaki `struct` örneğinde `yazar` ve `isbn`'i unutmanız gibi. Kural basit: elinizdeki her üyeyi, mümkünse, üye başlatma listesinde ilkleyin; gövdeyi yalnızca ilklemeden sonra gereken ek bir doğrulama ya da mantık için kullanın.

Bu ünitede gördüğünüz fikir kısaca şudur: kurucu bir nesneyi ilk andan itibaren tutarlı bir durumda kurar — nesnenin var olması ile doldurulması aynı, bölünemez adımdır — üye başlatma listesi bunu gövdedeki atamadan daha erken ve daha verimli bir anda yapar, çünkü üyeyi bir kere doğrudan kurar, önce boşaltıp sonra doldurmaz. `this` üye fonksiyona, o an hangi nesne üzerinde çalıştığını hatırlatan örtük işaretçidir; yıkıcı ise nesnenin ömrü bitince — kapsam dışına çıktığında — toparlanmayı örtük biçimde üstlenir. Gelecek ünitede bu ilkeleri `Kitap` sınıfının tamamını inşa ederken bir arada kullanacaksınız.

3 • Katılım — Şimdi Siz Deneyin

Bir `Kutu` sınıfının `private const int sabit_;` üyesi olsun. Kurucuyu gövdede atama ile — yani gövde içinde `sabit_ = deger;` yazarak — kurarsanız ne olur? Doğru biçim nedir?

Geribildirim. Bu kod derlenmez. Bir `const` üye yalnızca ilklenme anında, yani üye başlatma listesinde değer alabilir; kurucu gövdesindeki `sabit_ = deger;` satırı zaten ilklenmiş salt-okunur bir üyeye atama yapmaya çalışır (derleyici "assignment of read-only member" der). Doğrusu, değeri üye başlatma listesinde `: sabit_(deger)` biçiminde vermektir; gövde ise boş kalır. Aynı zorunluluk referans (`&`) üyeler için de geçerlidir: bir referans yalnızca kurulduğu anda bir nesneye bağlanır, sonradan atanamaz.

Kendinizi Sınayın 3.2

Üye başlatma listesi (`: baslik_(b)` gibi) ile kurucu gövdesinde atama (`baslik_ = b;`) arasındaki fark nedir? Yanıtınızda hem ilklenme *anını*, hem verimlilik farkını, hem de bu iki yoldan hangisinin `const` veya referans üyeler için zorunlu olduğunu ele alın.

5 • İzleme — Aktarım — Başka Nerede?

"Nesneyi ilk andan tutarlı kur" ilkesi her tipte geçerlidir: Kitaplık projesinde ileride bir `Uye` kaydı da eksik veya geçersiz bir durumda "doğmamalı", kurucusu üye numarasını ve adını daha ilk anda tutarlı biçimde kurmalıdır. Gündelik karşılığı, kütüphaneye yeni bir kitap kaydı açarken doldurulan kayıt formudur: eksik ya da tutarsız bir form sisteme hiç girmez; kurucu da nesneyi "kayıtsız" bir ara durumda bırakmaz. Kendi kaynağına sahip bir üye eklendiğinde ise — gelecek haftanın konusu — yıkıcının rolü, bu haftaki yüzeysel değinisinden çok daha belirleyici hâle gelecektir.

3.3 `Kitap` Sınıfını İnşa Etmek

1 • Ön-öğretim — Neden Önemli?

Ünite 1–2'de parçaları — erişim kontrolü, kurucu — ayrı ayrı gördünüz; şimdi bunları gerçek bir `Kitap` sınıfında birleştirmeniz gerekiyor. Peki `yil_` alanına dışarıdan `-5` gibi geçersiz bir değer atanmasını nasıl engellersiniz? Doğrulama mantığını (`gecerliMi`) sınıfın dışında serbest bir fonksiyon olarak bırakırsanız, onu çağırmayı unutan her kod nesneyi bozabilir. Bu ünite, kuralı nesneyle birlikte taşıyan — doğrulamayı sınıfın içine koyan — somut tasarımı kurar.

1 • Ön-öğretim — Ön Bilgi

Bu üniteye girmeden önce şunları hatırlayın: Ünite 1'in `public`/`private` ayrımı ve kapsülleme ilkesi; Ünite 2'nin parametreli kurucusu, üye başlatma listesi ve `this` işaretçisi. Hafta 2'nin `std::string` kullanımı ile `const` doğruluğu sezgisi

de — getter'ların neden `const` işaretlendiğini anlamak için — işinize yaracaktır.

Önceki iki ünitede `Kitap` sınıfının parçalarını tek tek gördünüz: `private` alanlarla kapsülleme, kurucuyla tutarlı bir başlangıç, üye başlatma listesiyle verimli ilkleme, yıkıcıyla düzenli bir kapanış. Bu ünitede bu parçaları bir araya getirip `Kitap` sınıfının tamamına bakıyorsunuz — artık tek tek, birbirinden kopuk örnekler değil, kütüphanenin gerçek kod tabanında duran, derlenebilir sınıfın kendisi. Aşağıda göreceğiniz her kod kesiti doğrudan `kod_ornekleri/hafta03/kitap.hpp` ve `kitap.cpp`'den alınmıştır; bu yüzden burada okuduğunuz şey bir soyutlama değil, derlenip çalışan gerçek sınıfın kendisidir.

3.3.1 Private alanlar: sınıfın verisi

`Kitap` sınıfının taşıdığı veri dört alandan oluşur; hepsi `private` bölümde durur, sınıf dışından hiçbir kod bunlara doğrudan erişemez:

```
16    std::string baslik_;
17    std::string yazar_;
18    std::string isbn_;
19    int yil_;
```

Dört alandan üçü (`baslik_`, `yazar_`, `isbn_`) `std::string` tipindedir — yani değişken uzunlukta metin tutarlar; dördüncüsü (`yil_`) temel bir `int`'tir. Tümü aynı `private:` etiketinin altında, yan yana durur; bu, `Kitap` nesnesinin bellekte taşıdığı verinin *tamamının* bu dört satırla tanımlandığı anlamına gelir — sınıfın başka hiçbir gizli alanı yoktur.

Her alanın adının sonunda bir alt çizgi görüyorsunuz: `baslik_`, `yazar_`, `isbn_`, `yil_`. Bu, kod tabanı boyunca tutarlı uygulanan bir adlandırma kuralıdır ve rastgele bir tercih değildir — somut bir sorunu çözer. Bu sorunu görmek için Ünite 2'deki kurucuyu hatırlayın: kurucunun dört parametresi de — `baslik`, `yazar`, `isbn`, `yil` — tam olarak alan adlarının alt çizgisiz hâlidir; üye başlatma listesi `: baslik_(baslik), yazar_(yazar), isbn_(isbn), yil_(yil)` biçiminde yazılmıştı. Şimdi alt çizgi kuralı hiç olmasaydı ne olacağını düşünün: alanı da parametreyle tamamen aynı adla, `baslik` olarak tanımlamış olurdunuz. Böyle bir durumda kurucu gövdesine `baslik = baslik;` yazmak size hiçbir şey kazandırmazdı — iç içe kapsamda aynı ad kullanıldığında derleyici en yakın (yerel, yani parametre) bağlamı kazanır kabul eder, dolayısıyla bu satır parametreyi kendi üzerine atar, üyeye hiç dokunmaz. Tam bu tuzağı Ünite 2'de `setYil(int yil)` örneğinde somut olarak gördünüz: sol taraftaki üyeyi sağ taraftaki parametreden ayırmak için `this->yil_ = yil;` yazmanız, yani `this` işaretçisine açıkça başvurmanız gerekmişti. Alt çizgi kuralı bu ayrımı bir dilbilgisi zorunluluğu (her yerde `this->` yazma zorunluluğu) olmaktan çıkarıp, gözle görülür bir isim farkına çevirir: `baslik_` ile `baslik` hiçbir zaman karışmaz, `this->` yazmasanız bile hangisinin üye, hangisinin parametre olduğu satıra bakar bakmaz bellidir.

Alanların tümünü `private` tutmanızın gerekçesi ise Ünite 1'den tanıdık: kapsülleme. Bir alan `public` olsaydı, kod tabanındaki herhangi bir yer `yil_`'e eksi bir sayı ya da `isbn_`'e anlamsız bir dizge atayabilirdi — sınıfın kendisi buna itiraz edemezdi. Nitekim Ünite 1'deki `k.yil = -3;` sorununun tıpatıp aynısı burada da geçerlidir: `yil_` `public` olsaydı, kod tabanının herhangi bir köşesindeki bir satır `k.yil_ = -3;` yazıp bu değeri sessizce bozabilirdi; sonucu ancak günler sonra, o değeri okuyan bambaşka bir fonksiyon çalıştığında görürdünüz. `private` yapmak, bu dört alana yalnızca `Kitap` sınıfının kendi üye fonksiyonları üzerinden dokunulabileceğini —

ve bu bozuk atamanın daha derleme aşamasında imkânsız kılındığını — garanti eder.

3.3.2 Public arayüz: sınıfın davranışı

arayüz: bir sınıfın dışarıya açtığı public üye fonksiyonlar bütünü (*interface*); sınıfla dışarıdan etkileşimin tek yolu

Alanlar gizliyken, sınıfla etkileşiminiz tamamen aşağıdaki **arayüz** üzerinden yürür:

```
   Kitap(std::string baslik, std::string yazar, std::string isbn, int yil);
   const std::string& baslik() const { return baslik_; }
   void setYil(int yil);
   bool gecerliMi() const;
   void yazdir() const;
```

Bu beş bildirim, `Kitap` ile dışarıdan kurabileceğiniz her ilişkiyi kapsar; bunları sırayla izleyin. Kurucu zaten Ünite 2'den tanıdık — dört alanı üye başlatma listesinde ilkler. `baslik()` bir *getter*'dır: `baslik_` alanını değiştirmeden, yalnızca bir referans (`const std::string&`) olarak dışarı verir; başlığı kopyalamadan okumanıza izin verir ama üzerine yazmanıza izin vermez — dikkat ederseniz `yazar()` ya da `isbn()` gibi bir getter burada yok, çünkü bu haftaki tasarımda yalnızca başlığı dışarı açmak yeterli görülmüştür. `setYil(int yil)` ise kontrollü bir *setter*'dır: `yil_`'i doğrudan değil, sınıfın kendi kuralından geçirerek günceller. `gecerliMi()` ve `yazdir()` alanlara değer atamaz ama onları okur; ikisini de aşağıdaki iki bölümde ayrıntılı göreceksiniz.

Beş bildirimden üçü — `baslik()`, `gecerliMi()`, `yazdir()` — `const` ile bitiyor. `const` anahtar kelimesinin tam olarak ne söz verdiğini adım adım açın. Bir üye fonksiyon `const` olarak bildirildiğinde, derleyici o fonksiyonun gövdesi içinde nesnenin `this` işaretçisini artık sıradan `Kitap*` değil, `const Kitap*` olarak ele alır — yani fonksiyon nesneye hâlâ erişebilir, onu okuyabilir, ama hiçbir alanını değiştiremez; `this` üzerinden ulaşılan her alan da örtük olarak `const` sayılır. Kısacası `const` yazarak derleyiciye "bu fonksiyon nesneyi değiştirmeyecek" sözünü verirsiniz, derleyici de bu sözü sizin yerinize, gövdenin her satırında denetler.

Bu sözün derleyici tarafından nasıl zorla uygulandığını, `Kitap`'tan bağımsız küçük bir örnekle somutlaştırın:

```
class KitapOrnek {
public:
   void yazdirDeneme() const {
      yil_ = 1980; // DERLEME HATASI: const govde icinde uyeye yazilamaz
   }
private:
   int yil_;
};
```

Bu satır derlenmez. `yazdirDeneme() const` olarak bildirildiği için gövde içinde `this`, `const KitapOrnek*` tipindedir; `yil_ = 1980;` ifadesi ise gerçekte `this->yil_ = 1980;` demektir ve bu, `const` bir işaretçinin gösterdiği veriyi değiştirmeyi dener. Derleyici bunu, Ünite 2'de gördüğünüz `const` bir üyeye (`sabit_ = deger;`) atama girişimiyle tamamen aynı gerekçeyle reddeder: bir kere "değiştirmeyeceğim" sözü verilmiş bir bağlamda değiştirme girişimi, tip sisteminin izin vermediği bir işlemdir.

Şimdi bu kuralı `kitap.hpp`'deki beş bildirime tek tek uygulayın. `baslik()`, `gecerliMi()` ve `yazdir()` üçü de yalnızca alan okur, hiçbir alana yazmaz; bu yüzden `const` olarak bildirmek onlara hiçbir kısıtlama getirmez — zaten yapmayı düşünmedikleri bir şeyi yapmayacaklarına söz vermiş olurlar, karşılığında da bu üç fonksiyonu `const` bir `Kitap` nesnesi üzerinde bile çağırabilirsiniz. Kurucu ise tam tersini yapar: dört alanı da üye başlatma listesinde ilk kez değerle doldurur;

bir nesnenin var olması demek alanlarının ilklenmesi demektir, bu da nesneyi değiştiren (daha doğrusu var eden) bir işlemdir — kurucuyu `const` yapmak zaten anlamsız olurdu, çünkü kurucunun tek işi nesneyi (henüz var olma sürecindeyken) doldurmaktır. `setYil(int yil)` de benzer şekilde `yil_` alanına yeni bir değer yazar; onu `const` yapsaydınız gövdesindeki `if (yil > 0) yil_ = yil;` satırı — tam olarak yukarıdaki `KitapOrnek` örneğindeki gibi — derlenmezdi. Kurucu ile `setYil()` bu yüzden `const` değildir: ikisi de alanlara değer atayarak nesnenin durumunu gerçekten değiştirir, bu yüzden böyle bir söz veremezler.

3.3.3 Doğrulama neden sınıfın içinde yaşar?

`gecerliMi()`'nin gövdesine bakın:

```
11 bool Kitap::gecerliMi() const {
12     return !baslik_.empty() && isbn_.size() == 13 && yil_ > 0;
13 }
```

Üç koşul bir arada kontrol edilir: başlık boş olmamalı, ISBN tam 13 karakter uzunluğunda olmalı (standart ISBN-13 biçimine kabaca bir yaklaşım), yıl sıfırdan büyük olmalı. Burada durup şunu sorabilirsiniz: bu kontrolü neden `Kitap` sınıfının *içine* yazdınız — neden `bool kitapGecerliMi(const Kitap& k)` gibi sınıf dışında, serbest bir fonksiyon olarak değil? Yanıt yine kapsüllemedir. Bu üç kural — boş olmayan başlık, doğru uzunlukta ISBN, pozitif yıl — `Kitap`'ın *değişmezini* (*invariant*), yani her an doğru kalması gereken kuralları tanımlar; bu kurallar sınıfın kendi sorumluluğudur, sınıfı kullanan kodun sorumluluğu değildir.

Bu iddiayı soyut bırakmayın; `gecerliMi()`'yi gerçekten serbest bir fonksiyon olarak yazmayı deneseydiniz ne olurdu, satır satır izleyin:

```
1 bool kitapGecerliMi(const Kitap& k) {
2     return !k.baslik_.empty() && k.isbn_.size() == 13 && k.yil_ > 0;
3     // ^^^^^^^^^^ ^^^^^^^^ ^^^^^
4     // uc alana da dogrudan erisim gerekir -- ama uculu de private
5 }
```

Bu fonksiyon `kitap.hpp`'nin dışında, hiçbir üye fonksiyon olmadan yazılmaya çalışılsaydı, `k.baslik_`, `k.isbn_` ve `k.yil_` ifadelerinin üçü de derlenmezdi — çünkü bu üç alan `private`'tır ve yalnızca `Kitap`'ın kendi üye fonksiyonları onlara erişebilir (Ünite 1'i hatırlayın). Bu satırı derletebilmenin tek yolu üç alanı da `public` yapmak olurdu; ama bunu yaptığınız an, bu ünitenin başında kurduğunuz kapsülleme tamamen ortadan kalkar — artık yalnızca `gecerliMi()` değil, kod tabanındaki her satır bu üç alana çıplak erişebilir, dolayısıyla Ünite 1'de gördüğünüz `k.yil = -3;` sorunu aynen geri döner. Alanları `public` yapmadan bu doğrulamayı serbest fonksiyonla yazmanın tek yolu ise `baslik()` gibi `isbn()` ve `yil()` için de ayrı getter'lar eklemek olurdu; ama bu kez de doğrulama kuralının *kendisi* sınıfın dışına, `kitap.hpp`'yi kullanan herhangi bir dosyaya taşınmış olur — her geliştirici, yeni bir `Kitap` nesnesi kullanmadan önce bu serbest fonksiyonu çağırmayı hatırlamak zorunda kalır. Birileri unutursa, ya da yeni bir dosyada bu fonksiyonun var olduğunu hiç bilmezse, geçersiz bir `Kitap` nesnesi hiçbir uyarı vermeden kod tabanında dolaşmaya devam eder — tıpkı Ünite 1'in başındaki `struct` örneğinde gördüğünüz gibi, hatanın *doğduğu* yer ile hatanın *fark edildiği* yer yine birbirinden kopar. Üye fonksiyon olarak tutmak, doğrulama kuralının sınıfla birlikte seyahat etmesini sağlar: `Kitap` nerede kullanılırsa kullanılsın, kendi geçerliliğini kendi tanımladığı tek bir yerden sorgulayabilir.

Analoji

Bir kütüphaneye yeni gelen bir kitabı düşünün: raflara konmadan önce kayıt masasındaki görevli başlığın boş olmadığını, ISBN'in doğru biçimde yazıldığını, basım yılının anlamlı olduğunu kontrol eder — bu kontrolü kütüphaneye uğrayan her okuyucu kendi başına, kendi bildiğince yapmaz; her zaman aynı görevli, aynı kurala göre yapar. `gecerliMi()` de tam olarak bu görevlidir: `Kitap` nesnesinin kendi geçerlilik kuralını taşıyan, sınıfın dışına çıkmayan tek bir denetim noktasıdır.

3.3.4 Nesneyi ekrana yazdırmak

Son üye fonksiyon, `Kitap` nesnesinin durumunu okunabilir biçimde ekrana basar:

```
17 void Kitap::yazdir() const {
18    std::cout << baslik_ << " — " << yazar_
19              << " (" << yil_ << "), ISBN: " << isbn_ << '\n';
20 }
```

`yazdir()` de `gecerliMi()` gibi yalnızca okur, hiçbir alana yazmaz — bu yüzden o da `const` olarak bildirilmiştir. Dört alanın tümünü tek bir satırda, insanın okuyacağı bir biçimde bir araya getirir; başlık, yazar, yıl ve ISBN'i sınıfın dışına, doğrudan alan erişimi olmadan, yalnızca bu üye fonksiyon üzerinden aktarır.

Aşağıdaki şekil, kurucu çağrıldıktan sonra somut bir `Kitap` nesnesinin belleğinde alanların hangi değerlerle dolu olduğunu gösteriyor:



Şekil 3.2: Bir `Kitap` nesnesinin alanları ve değerleri.

Karmaşıklık (big-O)

`baslik()` gibi bir getter ya da `setYil()` gibi bir setter, tek bir alana doğrudan erişip okur ya da yazar. Bunun neden sabit zamanlı olduğunu düşünün: her alanın nesne içindeki konumu (bellek ofseti) derleme anında bellidir, dolayısıyla `baslik_`'e ulaşmak nesnenin başlangıç adresine sabit bir ofset eklemek-

ten ibarettir — ne bir döngü, ne bir arama gerekir. Bu yüzden karmaşıklık nesnenin kaç alan taşıdığından ya da bir alanın (ör. `baslik_` dizgesinin uzunluğu) ne kadar büyük olduğundan bağımsız, sabit zamanda çalışır: $O(1)$.

Bu ünitede gördüğünüz fikir kısaca şudur: `Kitap` sınıfı, önceki iki ünitede parça parça öğrendiğiniz her ilkeyi tek bir tanımda birleştirir — `private` alanlar veriyi korur, `public` arayüz kontrollü bir erişim sunar, kurucu nesneyi tutarlı doğurur, `gecerliMi()` sınıfın kendi değişmezini kendi içinde denetler. Gelecek ünitede bu sınıfı `.hpp`/`.cpp` dosyalarına ayırıp bir `namespace` içine yerleştirerek gerçek bir kütüphane parçası hâline getireceksiniz.

3 • Katılım ⌨ Şimdi Siz Deneyin

`Kitap` sınıfında `baslik_` alanını döndüren bir `baslik()` getter'ı var; dönüş tipi `const std::string&`, imzasının sonu ise `const`. Buna benzer biçimde, `yazar_` alanını döndüren ve nesneyi değiştirmeyen bir `yazar()` üye fonksiyonu yazın.

Geribildirim. İmza şöyledir: dönüş tipi `const std::string&`, fonksiyon adı `yazar`, parametre listesi boş ve imzanın sonunda `const` bulunur; gövde ise yalnızca `return yazar_;` satırından oluşur. İmzanın sonundaki `const`, fonksiyonun nesneyi (ve `yazar_`'ı) değiştirmediğini garanti eder; bu sayede `const` bir `Kitap` üzerinde de çağrılabilir. Dönüş tipinin `const std::string&` olması ise `yazar_` içeriğinin gereksiz yere kopyalanmasını önler.

Kendinizi Sınayın 3.3

`gecerliMi()` neden `Kitap` sınıfının dışında serbest bir fonksiyon olarak değil, sınıfın içinde bir üye fonksiyon olarak durur? Yanıtınızda kapsülleme ilkesini ve sınıfın kendi değişmezini (invaryantını) kimin denetlemesi gerektiğini ele alın.

5 • İzleme ↗ Aktarım — Başka Nerede?

"private alan + doğrulayan public arayüz" kalıbı, Kitaplık projesinin bundan sonraki tüm sınıflarında yeniden kullanılır — bir `Uye`'nin üyelik durumu, bir `Odunc`'un iade tarihi de kendi geçerlilik kurallarını kendi içinde tutar. Gündelik karşılığı, otomat makinesinin stok ve kasa durumunu kendi içinde tutup dışarıya yalnız kontrollü tuş arayüzü açmasıdır: geçersiz bir işlem makinenin içine hiç ulaşamaz. Doğrulamayı verinin yanında tutan her tasarım, bu haftanın `Kitap` sınıfıyla aynı temel fikri paylaşır.

3.4 Kütüphaneyi Kurmak: Başlık/Kaynak ve İsim Uzayı

1 • Ön-öğretim 📣 Neden Önemli?

Şu ana kadar her şey tek bir dosyada duruyordu; ama Kitaplık dönem boyunca büyüyecek, yeni sınıflar eklenecek. Tüm sınıf tanımı ve gövdeleri tek dosyada kalırsa, küçük bir gövde değişikliği bile projenin bu dosyayı gören her parçası-

nı yeniden derletir; ayrıca başka bir kütüphanedeki bir başka `Kitap` sizinkiyle çakışır. Bu ünite `.hpp`/`.cpp` ayrımı ve `namespace` ile bu ölçek sorununu çözer.

1 • Ön-öğretim **Ön Bilgi**

Bu üniteye girmeden önce şunları hatırlayın: Ünite 3'te tamamladığınız `Kitap` sınıfı (arayüz ile üye fonksiyon gövdeleri) ve Ünite 1'in kapsülleme fikrinin "arayüz/uygulama ayrımı" olarak büyümesi. Hafta 2'nin derleme/bağlama (*compile/link*) süreci sezgisi de — bir değişikliğin neyi yeniden derlettiğini anlamak için — işinize yarayacaktır.

Önceki ünitede `Kitap` sınıfının tamamını tek bir tanımda gördünüz — `private` alanlar, `public` arayüz, kurucu, `gecerliMi()`, `yazdir()`. O tanımı orada, kendi başına duran bir bütün olarak okudunuz; ama gerçek bir kod tabanında bir sınıf hiçbir zaman tek başına yaşamaz. Şunu düşünün: dönem ilerledikçe `kitaplik`'e yeni sınıflar eklenecek, her biri kendi `.cpp` dosyasında yaşayacak, ama çoğu bir noktada `Kitap` tipini kullanacak — bir ödünç kaydı bir `Kitap` nesnesine referans tutacak, bir arama fonksiyonu bir `Kitap` listesi tarayacak. Şimdi `Kitap` sınıfının tüm tanımını — alanlarını, kurucusunu, her üye fonksiyonun gövdesini — tek bir dosyada tuttuğunuzu ve dönem boyunca yazacağınız her yeni `.cpp` dosyasının bu tanımı kullanabilmek için o dosyanın tamamını yeniden okumak, hatta yeniden derlemek zorunda kaldığını hayal edin: sınıfı kullanan her dosya, üye fonksiyonların gövdesini — yani nasıl çalıştıklarının her ayrıntısını — de gereksiz yere görmüş, derleyicinin de gereksiz yere yeniden işlemiş olur. Bu ünitede `Kitap` sınıfını iki dosyaya ayırıp derlenebilir, tekrar kullanılabilir bir kütüphane parçasına dönüştürüyorsunuz.

3.4.1 `.hpp` ve `.cpp`: bildirim ile tanımın ayrılması

başlık dosyası: bir sınıfın ya da fonksiyonun bildirimlerini — neyin var olduğunu, hangi imzayla çağrıldığını — barındıran, gövde içermeyen dosya (*header file*); geleneksel uzantısı `.hpp`.

C++'ta bir sınıfı anlatmanın iki farklı katmanı vardır: sınıfın *neler sunduğu* ile üye fonksiyonların *nasıl çalıştığı*. Birinci katman geleneksel olarak bir **başlık dosyası** içine, ikinci katman ise bir *kaynak dosyasında* (*source file*, `.cpp`) yaşar. `kitap.hpp` dosyasının genel iskeletine bakın — sınıfın gövdesini Ünite 3'te parça parça zaten kurdunuz, burada yalnızca dosyanın dış hatları var:

```
#pragma once
#include <string>

namespace kitaplik {
class Kitap { /* alanlar ve arayuz: Unite 3'te kurdunuz */ };
} // namespace kitaplik
```

Bu dosyada üye fonksiyonların gövdesi yok — yalnızca imzalar var: kurucunun hangi parametreleri aldığı, `gecerliMi()`'nin `bool` döndürdüğü ve `const` olduğu, alanların hangi tipte olduğu. `kitap.cpp` dosyası ise aynı fonksiyonların gerçek gövdelerini — `gecerliMi()`'nin üç koşulu nasıl kontrol ettiğini, kurucunun üye başlatma listesini — barındırır; bu gövdeleri Ünite 2 ve Ünite 3'te zaten gördünüz.

Bu ayrımın kazandırdığı ilk şeyi — *derleme birimi* kazancını — somut bir senaryoyla izleyin. Diyelim ki `gecerliMi()`'nin gövdesini `kitap.cpp` içinde değiştiriyorsunuz — örneğin ISBN uzunluk kontrolünü on üç yerine on karaktere de izin verecek şekilde gevşetiyorsunuz. Derleyici yalnızca `kitap.cpp` dosyasını yeniden okur ve yeni bir nesne dosyası (*object file*, `kitap.cpp.o`) üretir; `kitap.hpp` hiç değişmediği için `kitap_demo.cpp`'nin — ya da dönem boyunca `kitap.hpp`'yi include eden başka herhangi bir `.cpp` dosyasının — yeniden derlenmesine gerek yoktur;

bağlayıcı (*linker*) yalnızca yeni `kitap.cpp.o`'yu eskisinin yerine koyup diğer nesne dosyalarıyla birlikte bağlar. Şimdi tam tersini düşünün: bu kez `kitap.hpp`'ye yeni bir `public` üye fonksiyon ekliyorsunuz — örneğin `yazarGetir()` adlı bir getter. Bu değişiklik `kitap.hpp`'nin *içeriğini* değiştirdiği için, bu dosyayı include eden *her* `.cpp` dosyası — `kitap_demo.cpp`, `kitap.hpp`'yi kendisi de include eden `kitap.cpp`, dönem boyunca ekleyeceğiniz her yeni dosya — yeniden derlenmek zorundadır; çünkü önişlemci her birinin içine `kitap.hpp`'nin güncel hâlini yeniden yapıştırır, derleyici de bu güncel hâli görmeden eski nesne dosyalarını güvenle kullanamaz. Kısacası: bir `.cpp` değişikliği ucuzdur (yalnızca o dosya yeniden derlenir), bir `.hpp` değişikliği ise pahalıdır (o başlığı kullanan her dosya yeniden derlenir) — bu da başlık dosyalarını olabildiğince kararlı tutmanın, gövdeleri ise rahatça değiştirebilmenin pratik nedenidir. Bu tek kazancın yanına iki şey daha eklenir: (2) *tekrar kullanım* — aynı `kitap.hpp` dosyasını, dönem boyunca yazacağınız her yeni `.cpp` dosyası aynı bildirimlerle include edebilir, sınıfı sıfırdan yeniden tanımlamanız gerekmez; (3) *arayüz/uygulama ayrımı* — sınıfı kullanan kod yalnızca *ne* yapabildiğini bilmesi gerekir, *nasıl* yapıldığını bilmesi gerekmez.

Analoji

Header/source ayrımını bir kütüphanenin kataloğu ile rafı arasındaki ayrıma benzetebilirsiniz. Katalog (`.hpp`) hangi kitapların var olduğunu, hangi konuda olduklarını, nerede durduklarını listeler — kısa, taranabilir bir özet. Rafın kendisi (`.cpp`) ise gerçek içeriği, kitapların sayfalarını barındırır. Kataloğa bakan bir okuyucu, rafın iç düzenini — kitapların fiziksel olarak nasıl dizildiğini — bilmek zorunda değildir; yalnızca kataloğun söylediğine güvenip ilgili kitabı ister. `Kitap` sınıfını kullanan bir `.cpp` dosyası da tam olarak böyle davranır: `kitap.hpp`'yi include eder, hangi üye fonksiyonların var olduğunu görür, gövdelerin nasıl yazıldığını hiç bilmeden onları çağırır. Kataloğu güncellemek (yeni bir kitap girişi eklemek) kütüphanenin tamamını yeniden düzenlemeyi gerektirmez; ama rafların fiziksel yerleşimini baştan değiştirmek — yani katalogda kayıtlı *yerlerin kendisini* değiştirmek — kataloğa bakan her okuyucunun bilgisini de güncel tutmayı gerektirir. Az önce gördüğünüz `.cpp`/`.hpp` maliyet farkı tam olarak bu ayrıma karşılık gelir.

Yukarıdaki iskeletin en tepesindeki `#pragma once` satırını fark etmişsinizdir. Bu satırın çözdüğü sorunu soyut bırakmayın, küçük bir senaryoyla somutlaştırın. Önişlemcinin `#include` karşısında yaptığı iş aslında çok basittir: `#include "kitap.hpp"` satırının yerine, `kitap.hpp` dosyasının *tüm içeriğini* olduğu gibi yapıştırır — tıpkı bir metin editöründe kopyala-yapıştır yapmış gibi. Şimdi düşünün ki dönem ilerledikçe `kitaplik`'e `katalog.hpp` adlı yeni bir başlık dosyası eklediniz ve bu dosya, kendi bildirimlerinde `Kitap` tipine ihtiyaç duyduğu için kendisi de bir `#include "kitap.hpp"` satırı içeriyor. `kitap_demo.cpp` dosyanız hem doğrudan `#include "kitap.hpp"` hem de `#include "katalog.hpp"` yazarsa, önişlemci `kitap.hpp`'nin içeriğini bu tek derleme birimine *iki kez* yapıştırır: bir kez doğrudan, bir kez de `katalog.hpp` üzerinden dolaylı olarak. Sonuç, aynı `class Kitap` tanımının aynı dosyada iki kez görünmesidir ve derleyici buna "yeniden tanımlama" (*redefinition*) hatasıyla itiraz eder — üstelik bu hataya, elle iki kez `#include` yazmadan da, yalnızca başlıklar birbirini include ettiği için, farkında olmadan yol açabilirsiniz. `#pragma once` derleyiciye "bu dosyayı bir derleme biriminde yalnızca bir kez oku" der: önişlemci `kitap.hpp`'yi bir kez yapıştırdıktan sonra, aynı derleme birimi içinde bu dosyaya gelen her sonraki `#include` satırını sessizce atlar, dolayısıyla `class Kitap` tanımı hiçbir zaman ikinci kez yapıştırılmaz. Satırın

`#` ile başlaması tesadüf değildir: bu bir C++ ifadesi değil, kaynak kodu derleyiciye gitmeden önce işleyen *önişlemciye* (*preprocessor*) verilmiş bir talimattır.

3.4.2 `namespace kitaplik`: kütüphanenin ilk sınıfı

Yukarıdaki iskelete yeniden bakarsanız, `class Kitap` tanımının çıplak durmadığını, bir `namespace kitaplik` bloğunun içine yerleştirildiğini görürsünüz. Bunun çözdüğü sorunu da somut bir senaryoyla düşünün. Diyelim ki dersin başka bir alıştırmasında ya da paylaştığınız bir kod parçasında, sizin yazmadığınız ayrı bir modül — örneğin bir mağaza/envanter örneği — kendi amaçlarına uygun, tamamen farklı alanlar taşıyan bir `class Kitap` daha tanımlıyor ve siz bir noktada iki başlık dosyasını da (`kitap.hpp` ve o örneğin kendi başlığını) aynı `.cpp` dosyasında include etmek zorunda kalıyorsunuz — ortak bir yardımcı dosya yazmak ya da iki alıştırmayı bir araya getirmek gibi bir durumda bu hiç de uzak bir ihtimal değildir. Her iki `Kitap` da isim uzayı olmadan, doğrudan global kapsamda tanımlanmış olsaydı, önişlemci her iki başlığı da aynı derleme birimine yapıştırdığında derleyici aynı adı taşıyan iki farklı `class Kitap` tanımıyla karşılaşır — tıpkı `#pragma once` bölümünde gördüğünüz "yeniden tanımlama" hatasına çok benzer bir çakışma, ama bu kez tek bir dosyanın iki kez include edilmesinden değil, iki *farklı* dosyanın aynı kısa adı kullanmasından doğar. Bir **isim uzayı** içine yerleştirmek bu çakışmayı baştan imkânsız kılar: `Kitap` adını `kitaplik` isim uzayına sabitlersiniz; sınıfa dışarıdan erişmek istediğinizde — tıpkı `kitap_demo.cpp` dosyasında gördüğünüz gibi — `kitaplik::Kitap` yazarsınız. Öteki `Kitap` kendi isim uzayına (örneğin `magaza::Kitap`) yerleştirilmiş olsaydı, iki tip aynı kısa adı taşısa bile derleyici bunları asla karıştırmaz, çünkü tam nitelenmiş adları — `kitaplik::Kitap` ile `magaza::Kitap` — birbirinden tamamen farklıdır.

isim uzayı: birden çok sınıf, fonksiyon ya da değişkeni ortak bir ad altında gruplayıp, aynı adın farklı isim uzaylarında çakışmadan var olabilmesini sağlayan sözdizimsel kapsayıcı (*namespace*).

Bu, yalnızca teknik bir ayrıntı değil, bu dönemin anlatı çerçevesinin de tam kalbidir: `Kitap`, `kitaplik` isim uzayına yerleştirdiğiniz *ilk* sınıftır — ama sonuncusu olmayacaktır. Dönem boyunca aynı isim uzayına yeni sınıflar ekleyeceksiniz; her biri bu ünitede gördüğünüz kalıbı tekrar eder — kendi `.hpp`/`.cpp` çiftinde yaşar, en tepesinde `#pragma once`, sınıf tanımının etrafında yine `namespace kitaplik { ... }`, `CMakeLists.txt`'nin kaynak listesine eklenmiş bir satır — ve hepsi aynı `kitaplik` çatısı altında toplanır. Bu yaklaşım DRY ilkesinin (*Don't Repeat Yourself* — kendinizi tekrar etmeyin) kütüphane ölçeğindeki karşılığıdır: her yeni sınıfı sıfırdan, izole bir dosyada, kendi adlandırma kurallarıyla değil; ortak bir isim uzayı, ortak bir derleme düzeni ve önceki sınıflarla tutarlı bir dosya yapısı içinde inşa edersiniz. Bu ünitede gördüğünüz `.hpp`/`.cpp` ayrımı ve `namespace kitaplik` bloğu, yalnızca `Kitap` için değil, dönem boyunca `kitaplik`'e ekleyeceğiniz her sınıf için aynı şekilde geçerli olan bir kalıptır.

3.4.3 CMake ile derlemek

`kitap.hpp` ve `kitap.cpp` artık ayrı dosyalar olduğuna göre, bunları — ve onları kullanan `kitap_demo.cpp`'yi — tek bir çalıştırılabilir dosyada bir araya getirecek bir derleme adımına ihtiyaç vardır. Kütüphaneniz büyüdükçe bu derleme komutunu elle, dosya dosya yazmak sürdürülemez hale gelir; bunun yerine `CMakeLists.txt` adlı bir yapılandırma dosyasıyla **CMake** kullanırsınız. Bu haftanın `CMakeLists.txt` dosyası kısa ve okunaklıdır: `cmake_minimum_required` ile gereken en düşük CMake sürümünü, `project(...)` ile proje adını bildirir; `set(CMAKE_CXX_STANDARD 20)` derleyiciye C++20 kullanmasını söyler; `add_executable(kitap_demo kitap_demo.cpp kitap.cpp)` ise `kitap_demo` adlı çalıştırılabilir dosyanın hangi kaynak dosyalarından — `kitap_demo.cpp` ve

CMake: C++ projelerinin hangi dosyalardan, hangi standartla derleneceğini betimleyen, platformdan bağımsız bir yapılandırma aracı (*build system generator*).

`kitap.cpp` — derleneceğini belirtir. Bu listeyi unutmanın somut bir bedeli vardır: `kitap.cpp` kaynak listesinden çıkarılsaydı, `kitap_demo.cpp` yine derlenirdi — çünkü derleyici yalnızca `kitap.hpp`'deki bildirimlere bakar, gövdelere ihtiyaç duymaz — ama bağlayıcı, çağrılan üye fonksiyonların gövdelerini hiçbir nesne dosyasında bulamayacağı için "tanımsız sembol" (*undefined reference*) hatasıyla durur. Kütüphaneniz büyüdükçe her yeni `.cpp` dosyasını bu listeye eklemeniz gerekir — CMake'in asıl kazandırdığı şey de budur: dosya sayısı arttıkça elle yazacağınız derleme komutu uzayıp karmaşıklaşırken, `CMakeLists.txt` kısa ve okunaklı kalır. Bu dersin kapsamında CMake'i yalnızca yüzeysel düzeyde tanıyorsunuz; asıl önemli olan, birden çok `.cpp` dosyasının tek bir çalıştırılabilir dosyada nasıl bir araya geldiğini görmenizdir.

Aynı derlemeyi CMake'siz, doğrudan derleyiciyi çağırarak da yapabilirsiniz — bu haftaki tek dosyalı ölçekte iki yaklaşım da aynı sonucu üretir:

Dene-Çalıştır

Aynı klasörde şu komutu çalıştırın:

Terminal

```
$ clang++ -std=c++20 kitap_demo.cpp kitap.cpp -o kitap_demo
```

ardından üretilen `kitap_demo` çalıştırılabilir dosyasını çalıştırın. Ekranda şu satırı görmeniz gerekir:

Beklenen Çıktı

```
Nutuk — Mustafa Kemal (1927), ISBN: 9789751000000
```

Bu komutu adım adım okuyun: `clang++` önce `kitap_demo.cpp` ve `kitap.cpp` dosyalarının her birini ayrı ayrı derleyip birer nesne dosyasına çevirir, sonra bu iki nesne dosyasını — ve gövdesini kullandıkları birbirlerinin bildirimlerini — bağlayıcı aracılığıyla tek bir çalıştırılabilir dosyada (`kitap_demo`) birleştirir; tam da `CMakeLists.txt`'deki `add_executable` satırının yaptığı şeydir, yalnızca CMake bu iki adımı (derleme ve bağlama) sizin için otomatik sıralar ve dosya sayısı büyüdükçe komutu elle yazmanızı gerektirmez. Bu çıktının nereden geldiğini `kitap_demo.cpp` dosyasında görüyorsunuz:

```
5  kitaplik::Kitap k{"Nutuk", "Mustafa Kemal", "9789751000000", 1927};
6  if (k.gecerliMi()) k.yazdir();
```

`main` fonksiyonu `kitaplik::Kitap` tipinden bir nesne kurar — isim uzayı öneki burada tam olarak neden gerekli olduğunu gösterir — sonra `gecerliMi()` ile nesnenin tutarlı olup olmadığını kontrol eder, yalnızca geçerliyse `yazdir()`'i çağırır. Bu üç satır, önceki üç ünitede tek tek inşa ettiğiniz her parçayı — kapsülleme, kurucu, doğrulama, isim uzayı — gerçek, derlenebilir bir programda bir araya getirir.

İpucu

`Kitap` nesneleri şu an güvenle kuruluyor ve yıkılıyor, ama Ünite 2'de gördüğünüz gibi bu haftanın yıkıcısı yüzeyseldi: `Kitap` yalnızca kendi kaynaklarını kendisi yöneten `std::string` ve `int` üyeler taşıyor. Gelecek hafta bu sınıfa kopyalama davranışı ve doğrudan bellek/kaynak yönetimi eklendiğinde — akıllı işaretçilerle — yıkıcının rolü çok daha belirleyici hale gelecek. Bu haftanın

`kitaplik` isim uzayı ve `.hpp`/`.cpp` düzeni, o genişlemenin üzerine oturacağı zemindir.

Bu ünitede gördüğünüz fikir kısaca şudur: bildirimi (`.hpp`) tanımdan (`.cpp`) ayırmak arayüzü uygulamadan koparır, derleme birimini küçültür ve tekrar kullanımı mümkün kılar; `#pragma once` bu ayrımı çoklu include'a karşı güvenceye alır; `namespace kitaplik` ise `Kitap`'ı — dönem boyu büyüyecek kütüphanenizin ilk sınıfını — isim çakışmasından korur; `CMakeLists.txt` ise bu büyüyen dosya kümesini tek bir okunaklı komutla derlenebilir tutar. Gelecek hafta bu sınıfa kopyalama ve kaynak yönetimi davranışı eklenecek; `Kitap` yalnızca doğru kurulan ve yıkılan değil, doğru *kopyalanan* bir nesne haline gelecek.

3 • Katılım — Şimdi Siz Deneyin

`Kitap` sınıfı neden `kitap.hpp` (bildirim) ile `kitap.cpp` (tanım) olarak iki dosyaya ayrılır ve `namespace kitaplik` içine alınır? İki gerekçe verin.

Geribildirim. (1) **Arayüz/uygulama ayrımı:** `kitap.hpp` "ne var" listesini (katalog), `kitap.cpp` ise "nasıl çalışır" gerçeğini (raf) tutar; yalnız `kitap.cpp` değişirse yalnız o yeniden derlenir, `.hpp`'yi include eden diğer dosyalar yeniden derlenmez. (2) **İsim çakışması:** `namespace kitaplik`, dönem boyu eklenecek tüm sınıfları tek bir isim alanında toplayıp başka kütüphanelerdeki aynı adlarla — örneğin başka bir `Kitap` ile — çakışmayı önler.

Kendinizi Sınayın 3.4

`Kitap` sınıfının bildirimini (`.hpp`) tanımından (`.cpp`) neden ayırırsınız? Yanıtınızda derleme birimi, tekrar kullanım ve arayüz/uygulama ayrımı kavramlarını — katalog/raf analojisini kullanarak — ele alın.

5 • İzleme — Aktarım — Başka Nerede?

Bu `.hpp`/`.cpp` ve `namespace` düzeni, Kitaplık'a dönem boyu eklenecek her yeni sınıfta aynen tekrarlanan bir kurulum kalıbıdır; ölçeklenen her C++ projesinin standart iskeletidir. Gündelik karşılığı bir kütüphanenin kataloğu (ne var, ne işe yarar) ile rafın (gerçek içerik) ayrılmasıdır: kataloğa bakan, rafın iç düzenini bilmek zorunda kalmadan aradığını bulur. Bu haftadan sonra kuracağınız her sınıf için kurulumun kendisi artık tanıdık bir alışkanlık olacaktır.

Bölüm Özeti

Özet

Geçen hafta bellek ve fonksiyonlarla çalıştınız; bu hafta o fonksiyonları veriyle aynı çatı altında, kontrollü bir erişimle bir araya getirdiniz. İlk ünitede struct + serbest fonksiyon modelinin sınırını gördünüz: veri korunmasızdır, kod tabanındaki herhangi bir fonksiyon herhangi bir alana serbestçe erişip tutarsız bir duruma sokabilir; davranış veriden kopuktur. Buna karşılık sınıf, veriyi ve o veri üzerindeki davranışı bir arada tanımlar ve `public`/`private` eri-

şim belirleyicileriyle kimin neye dokunabileceğini sınıfın kendisi belirler — bu, kapsülleme ilkesidir: iç durumu gizleyip yalnız kontrollü bir arayüzden erişitirmek. İkinci ünite bir nesnenin nasıl güvenle doğduğunu ve öldüğünü işledi: kurucu, nesneyi ilk andan itibaren tutarlı bir durumda kurar; üye başlatma listesi bunu gövdedeki atamadan farklı — ve `const`/referans üyeler için zorunlu — bir ilklenme anında yapar; `this` işaretçisi üye fonksiyonun "kendi nesnem" dediği örtük referanstır; yıkıcı ise nesnenin ömrü bitince otomatik çağrılıp toparlanmayı üstlenir — bu son nokta, gelecek haftanın habercisidir. Üçüncü ünitede bu ilkeleri somutlaştırıp kütüphanenin ilk sınıfını, `Kitap`'ı, inşa ettiniz: `baslik_`, `yazar_`, `isbn_`, `yil_` private alanları ve kurucu, getter'lar, `setYil()`, `gecerliMi()`, `yazdir()`'dan oluşan public arayüzü. `gecerliMi()`'nin sınıfın *içinde* durmasının nedeni de aynı ilkeydi: doğrulama mantığı sınıfın invaryantına aittir, dışarıda serbest bir fonksiyon olarak durursa kapsülleme delinir. Dördüncü ünite bu sınıfı gerçek bir kütüphane parçasına dönüştürdü: bildirimi (`.hpp`) tanımdan (`.cpp`) ayırdınız, `#pragma once` ile çoklu include'a karşı korudunuz ve her şeyi `namespace kitaplik` içine yerleştirdiniz — `Kitap`, dönem boyunca büyüyecek bu kütüphanenin yalnızca ilk sınıfı. `Kitap` nesneleri artık güvenle kuruluyor ve yıkılıyor; ama yıkıcının bu haftaki yüzeysel değinisi bir soruyu açık bıraktı: bir nesne bellek, dosya gibi bir *kaynağa* sahipse bu kaynak nasıl güvenle yönetilir? Gelecek hafta bu soruyu — kaynak yönetimi ve akıllı işaretçiler ile — yanıtlayacaksınız.

Kendinizi Sınayın Cevapları

Soru (Ünite 1 — bir alanı neden private yaparsınız). Bir alanı `private` yapmanızın gerekçesi kapsüllemedir: sınıfın dışından gelen herhangi bir kodun o alana *doğrudan* ve *kontrolsüz* biçimde erişip yazmasını engellemek. Alan `public` kalsaydı, kod tabanındaki herhangi bir yer örneğin bir kitabın yayın yılına negatif bir değer atayabilir ya da ISBN'i geçersiz bir dizgeyle değiştirebilirdi — sınıfın kendisi buna itiraz edemezdi, çünkü değişiklik hiçbir üye fonksiyondan geçmeden doğrudan alana yazılırdı. Alanı `private` yapıp yalnız `setYil()` gibi kontrollü bir arayüzden erişim verdiğinizde, o arayüzün içine bir doğrulama kuralı ekleyebilirsiniz; böylece sınıf kendi tutarlılığını (invaryantını) kendisi garanti eder. Kısacası: `private`, "bu veriye yalnız sınıfın belirlediği kurallarla dokunabilirsiniz" demenin dilidir.

Soru (Ünite 2 — üye başlatma listesi vs kurucu gövdesinde atama). İki yaklaşım da üyeye bir başlangıç değeri verir, ama farklı anlarda ve farklı maliyetlerle. Üye başlatma listesi (`: baslik_(b)` gibi) üyeyi kurucu gövdesi *çalışmadan önce*, nesnenin ilklenme anında doğrudan kurar; gövdedeki bir atama (`baslik_ = b;`) ise üye zaten (varsayılan biçimde) ilklenmiş *olduktan sonra* ikinci bir adımda üzerine yazar — bu, basit tipler için gözle görülür bir fark oluşturmasa da nesne içeren üyelerde gereksiz bir ilk kurulumu ve ardından bir atamayı, yani iki işlemi bir işlem yerine yaptırır. Daha kritik olan şu: `const` üyeler ve referans üyeler hiç atanamaz, yalnız ilklenebilir; bu yüzden böyle bir üyeniz varsa üye başlatma listesi *zorunludur*, gövdede atama seçeneği bile yoktur. Kısacası üye başlatma listesi hem daha verimli hem de bazı üye türleri için vazgeçilmezdir; gövdede atama yalnızca ilklemeden sonra ek bir mantık (doğrulama, hesaplama) gerektiğinde tercih edilir.

Soru (Ünite 3 — gecerliMi() neden sınıfın içinde durur). `gecerliMi()`'nin `Kitap` sınıfının dışında serbest bir fonksiyon olarak değil, sınıfın içinde bir üye fonksiyon olarak durmasının nedeni yine kapsüllemedir. Doğrulama mantığı — başlığın boş olmaması, ISBN uzunluğunun tutarlı olması, yılın pozitif olması — `Kitap`'ın *invaryantını*, yani her zaman doğru olması gereken kuralları tanımlar; bu kurallar sınıfın kendi sorumluluğudur, sınıfı kullanan kodun sorumluluğu değildir. Eğer `gecerliMi()` sınıf dışında serbest bir fonksiyon olsaydı, önce private alanlara erişebilmesi için ya alanları `public` yapmanız (kapsüllemeyi tamamen terk etmeniz) ya da o fonksiyona özel bir ayrıcalık tanımanız gerekirdi; ayrıca her yeni geliştirici doğrulamayı çağırmayı unutabilir ya da farklı bir dosyada tutarsız bir kopyasını yazabilirdi. Üye fonksiyon olarak tutmak, doğrulama kuralının sınıfla birlikte seyahat etmesini ve tek bir yerde, sınıfın kendi tanımında yaşamasını sağlar.

Soru (Ünite 4 — .hpp/.cpp neden ayrılır). Bildirimi (`.hpp`) tanımdan (`.cpp`) ayırmanızın nedeni arayüz ile uygulamayı birbirinden koparmaktır — kütüphane kataloğu ile rafın ayrılması gibi. `.hpp` dosyası `Kitap` sınıfının *neler sunduğunu* (hangi alanlar, hangi üye fonksiyonlar) listeler; bu dosyayı include eden herhangi bir kod, `gecerliMi()`'nin gövdesinin nasıl yazıldığını bilmeden yalnız katalog bilgisiyle sınıfı kullanabilir. `.cpp` dosyası ise gerçek uygulamayı, yani rafın içeriğini barındırır ve yalnız bir kez derlenir. Bu ayrım pratikte üç şey kazandırır: (1) derleme birimi — `.cpp` değiştiğinde onu kullanan her dosyayı yeniden derlemeniz gerekmez, yalnız o birim yeniden derlenir; (2) tekrar kullanım — aynı `.hpp`'yi birden çok `.cpp` dosyası include edebilir; (3) arayüz/uygulama ayrımı — sınıfı kullanan kod yalnız *ne* yapıldığını bilmesi gerekir, *nasıl* yapıldığını bilmesi gerekmez. `#pragma once` bu kataloğun aynı derleme biriminde birden fazla kez okunup çakışma oluşturmasını önler; `namespace kitaplik` ise `Kitap` gibi genel bir adın kütüphanenizin dışındaki başka bir `Kitap` ile çakışmasını engeller.